claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_416cc0e5-a35\agent-a914cf88489ab3c71.jsonl`
- SHA-256: `121327797462da1e9f4c336db76364d5e64fbbc0ea3ac1ba5ef4d5a91981c2e0`
- Source modified: `2026-06-22T09:47:42+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:45:35.964Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

```
{
 "title": "Exception handler at line 485 will mask real exceptions if they occur before run_corpus returns",
 "file": "scripts/nightly_reelcount.py",
 "line": "485-489",
 "severity": "medium",
 "detail": "At lines 485-489, ANY exception from run_corpus() is caught and returns exit code 2 (operational failure). This is by design (line 487 comment: 'never a false pass'). However, if an exception is raised INSIDE a video's run_one_video() handler (e.g., during segmenter import at line 335), it will propagate to run_corpus() and be caught here. The regression detection never runs. This is intentional ? operational failures (missing config, import error, GPU OOM) should exit 2, not 1. But if a video's segmenter CRASHES with a determinism bug (e.g., RNG doesn't seed correctly), the crash could mask a real regression. Current mitigation: line 330 catches fetch errors; line 338 catches missing segmenter. Deep segmenter crashes are unlikely and would exit 2 (operational), not hide as pass.",
 "fix": "No fix strictly required (design is intentional). Optional: add a config option to convert operational errors to regression (exit 1) if the owner wants stricter failure handling, but this is not recommended for a nightly tripwire."
}
```


2. assistant (2026-06-22T09:45:38.456Z)

I'll analyze this finding by examining the code to verify whether the exception handler truly masks regression detection or if the design is appropriately intentional.

3. user (2026-06-22T09:45:39.584Z)

```
1       """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM nightly).
2
```

```
    Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a deployment serves
    it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does not change**
    vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score of CACHED
    trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real model*, so it
    needs a GPU ? hence it runs on an ephemeral GCP GPU VM (``deploy/gcp/nightly_regression/``), emails the
    owner the report, and self-deletes. It is **default-additive**: a tripwire, not the promotion gate.

    What it asserts / reports, per the owner's ask ("quality metrics + regression report + cost"):
      ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected highlight clips.
        Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so this is asserted
        **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-None, see §guard).
      ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match block from
        ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small tolerance.
      ? COST of the run ? VM uptime (or processing wall) × machine $/hr via ``backend.billing`` ? USD + INR.

    Extensible pattern (add a video = one manifest row + re-freeze):
      ``eval_baselines/reelcount_manifest.json``  ? the videos (gs:// / local / gdrive:// URIs) + segmenter
      ``eval_baselines/reelcount_baseline.json``  ? the frozen expected counts (regenerated, never hand-edited)

    Usage
    -----
    Freeze the baseline from the current configured+checkpointed engine (after an intended change, or first run):
        python scripts/nightly_reelcount.py --update-baseline

    Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails if --email):
        python scripts/nightly_reelcount.py --email

    Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality dropped) · 2 operational
    (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter changed ? re-freeze).
    """
    from __future__ import annotations

    import argparse
    import datetime as _dt
    import json
    import os
    import sys
    from dataclasses import dataclass, field
    from typing import Any, Dict, List, Optional, Tuple

    REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    if REPO_ROOT not in sys.path:
        sys.path.insert(0, REPO_ROOT)

    # billing is pure (typing only) ? safe to import at module top.
    from backend import billing  # noqa: E402
```

```
49
50      DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51      DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52      DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54      #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
55      QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
56      DEFAULT_QUALITY_TOL = 0.02     # quality is float/labeled ? a looser tripwire than the
exact reel-count
57      DEFAULT_FX_INR_PER_USD = 83.0
58      #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
59      DEFAULT_MACHINE_USD_PER_HR = 0.35
60
61
62      @dataclass(frozen=True)
63      class Tolerances:
64          quality_tol: float = DEFAULT_QUALITY_TOL
65
66          def to_dict(self) -> Dict[str, Any]:
67              return {"quality_tol": self.quality_tol}
68
69
70      # ---------------------------------------------------------------------------- #
71      # Pure cost math (wraps backend.billing ? no IO).
72      # ---------------------------------------------------------------------------- #
73      def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                      fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75          """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).
76
77          ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
78          otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
79          rate_per_sec = float(machine_usd_per_hr) / 3600.0
80          usage = {billing.WALL_SECONDS: float(billed_seconds)}
81          rates = {billing.WALL_SECONDS: rate_per_sec}
82          usd, breakdown = billing.compute_cost_usd(usage, rates)
83          return {
84              "billed_seconds": round(float(billed_seconds), 1),
85              "gpu_seconds": round(float(gpu_seconds), 1),
86              "machine_usd_per_hr": float(machine_usd_per_hr),
87              "usd": usd,
88              "inr": billing.to_inr(usd, fx_inr_per_usd),
89              "breakdown": breakdown,
90          }
91
92
93      # ---------------------------------------------------------------------------- #
94      # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
95      # ---------------------------------------------------------------------------- #
96      def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
97                            cost: Dict[str, Any]) -> Dict[str, Any]:
98          """Compact, comparable snapshot from per-video run results.
99
100         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
101         A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
never hides it)."""
102         per_video: Dict[str, Any] = {}
103         for r in run_results:
104             per_video[r["name"]] = {
105                 "reel_count": r.get("reel_count"),
```

```
                "ok": bool(r.get("ok", False)),
                "error": r.get("error"),
                "quality": r.get("quality"),
            }
        return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
                "n": len(per_video)}


    @dataclass
    class Finding:
        video: str
        metric: str                # "reel_count" | "error" | a quality key | "segmenter" |
"corpus"
        kind: str                  # "regression" | "improvement" | "stale"
        baseline: Optional[float] = None
        current: Optional[float] = None
        detail: str = ""

        def message(self) -> str:
            if self.kind == "stale":
                return f"[STALE] {self.metric}: {self.detail}"
            if self.metric == "error":
                return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
            b = "?" if self.baseline is None else f"{self.baseline:g}"
            c = "?" if self.current is None else f"{self.current:g}"
            tag = "REGRESSION" if self.kind == "regression" else "improvement"
            return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"


    @dataclass
    class NightlyResult:
        findings: List[Finding] = field(default_factory=list)
        added: List[str] = field(default_factory=list)
        removed: List[str] = field(default_factory=list)

        @property
        def regressions(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "regression"]

        @property
        def improvements(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "improvement"]

        @property
        def stale(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "stale"]

        def overall_status(self) -> str:
            if self.regressions:
                return "REGRESSION"
            if self.stale:
                return "STALE"
            return "PASS"

        def exit_code(self) -> int:
            if self.regressions:
                return 1
            if self.stale:
                return 4
            return 0


    def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
        """Diff current run vs frozen baseline.
```

```
169
170         * Segmenter changed ? STALE (numbers not comparable; re-freeze).
171         * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
172         * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
173           quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
174         """
175         res = NightlyResult()
176         if baseline.get("segmenter") != current.get("segmenter"):
177             res.findings.append(Finding("", "segmenter", "stale",
178                                         detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}"))
179             return res
180
181         b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
182         res.added = sorted(set(c_pv) - set(b_pv))
183         res.removed = sorted(set(b_pv) - set(c_pv))
184         if res.added or res.removed:
185             res.findings.append(Finding("", "corpus", "stale",
186                                         detail=f"added={res.added} removed={res.removed}"))
187
188         for v in sorted(set(b_pv) & set(c_pv)):
189             b, c = b_pv[v], c_pv[v]
190             # 1) pipeline must still succeed
191             if not c.get("ok", False) or c.get("reel_count") is None:
192                 res.findings.append(Finding(v, "error", "regression",
193                                             detail=str(c.get("error") or "no reel_count
produced")))
194                 continue
195             # 2) reel count ? EXACT
196             bc, cc = b.get("reel_count"), c.get("reel_count")
197             if bc is not None and cc is not None and cc != bc:
198                 res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
199             # 3) quality metrics ? tolerance (only if both sides have them)
200             bq, cq = b.get("quality") or {}, c.get("quality") or {}
201             for m in QUALITY_HIGHER:
202                 bv, cv = bq.get(m), cq.get(m)
203                 if bv is None or cv is None:
204                     continue
205                 if cv < bv - tols.quality_tol:
206                     res.findings.append(Finding(v, m, "regression", bv, cv))
207                 elif cv > bv + tols.quality_tol:
208                     res.findings.append(Finding(v, m, "improvement", bv, cv))
209         return res
210
211
212     # ------------------------------------------------------------------------ #
213     # Report formatting (pure).
214     # ------------------------------------------------------------------------ #
215     def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
216         if not qd or qd.get(key) is None:
217             return "?"
218         return f"{qd[key]:.3f}"
219
220
221     def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
222                       result: NightlyResult, date: str, head: str) -> str:
223         status = result.overall_status()
224         badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
225                  "STALE": "? STALE (re-freeze the baseline)"}[status]
226         cost = current.get("cost", {})
227         L: List[str] = [
228             f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
```

```
            "",
            f"**Status: {badge}**  ·  exit code {result.exit_code()}  ·  segmenter
`{current.get('segmenter')}`",
            "",
            f"- HEAD `{head}`  ·  baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",
            f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})**  ·  "
            f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
            "",
        ]
        if result.regressions:
            L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
+ [""]
        if result.stale:
            L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
            L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]

        # Per-video table.
        b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
        L += ["## Per-video", "",
              "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
              "|---|---|---|---|---|"]
        for v in sorted(set(b_pv) | set(c_pv)):
            b, c = b_pv.get(v, {}), c_pv.get(v, {})
            bc = b.get("reel_count")
            cc = c.get("reel_count")
            rc = f"{'?' if bc is None else bc}?{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
            bq, cq = b.get("quality"), c.get("quality")
            tf = f"{_q(bq, 'tol_f1')}?{_q(cq, 'tol_f1')}"
            f5 = f"{_q(bq, 'f1@0.5')}?{_q(cq, 'f1@0.5')}"
            vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else "?"
            L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
        L.append("")

        if result.improvements:
            L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
                + [f"- {f.message()}" for f in result.improvements] + [""]
        L += ["---",
              "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
              "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
        return "\n".join(L)


    # ------------------------------------------------------------------------ #
    # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
    # ------------------------------------------------------------------------ #
    def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
        """Load ground-truth rally intervals from a golden-features JSON (``gts`` key) or a
rallies CSV
        (``start,end`` columns / pairs). Returns None when no labels are configured."""
        if not labels_ref:
            return None
        from backend.storage import fetch
        local = fetch(labels_ref)
        if local.endswith(".json"):
            with open(local, encoding="utf-8") as fh:
                data = json.load(fh)
            return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
```

```
283         # CSV: header start,end (or first two numeric columns)
284         import csv
285         out: List[Tuple[float, float]] = []
286         with open(local, encoding="utf-8") as fh:
287             for row in csv.reader(fh):
288                 try:
289                     out.append((float(row[0]), float(row[1])))
290                 except (ValueError, IndexError):
291                     continue
292         return out
293
294
295     def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
296         """``(start, end)`` pairs from segmenter result dicts, tolerant of the two key
conventions in the
297         codebase (``start_time``/``end_time`` ? motion/DB shape ? or ``start``/``end``).
Used only for the
298         optional quality metrics; the reel COUNT is ``len(segs)`` regardless, so a key
mismatch degrades
299         quality scoring gracefully (skipped) without ever miscounting reels."""
300         out: List[Tuple[float, float]] = []
301         for s in segs:
302             start = s.get("start_time", s.get("start"))
303             end = s.get("end_time", s.get("end"))
304             if start is not None and end is not None:
305                 out.append((float(start), float(end)))
306         return out
307
308
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                       rerun_on_mismatch: bool = True,
311                       baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e:  # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                     "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                     "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
```

```
        def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
            random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
            db = Database(db_path)
            video_id = compute_video_id(local)
            t0 = time.monotonic()
            result = seg_cls(db, config).process_video(video_id, local)
            wall = time.monotonic() - t0
            if isinstance(result, Failure):
                return None, None, wall, getattr(result, "message", "segmenter failed")
            segs = result.value if hasattr(result, "value") else result
            intervals = _segments_to_intervals(segs)
            return len(segs), intervals, wall, None

        import tempfile
        total_wall = 0.0
        with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
            count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
            total_wall += wall
            if err is not None:
                return {"name": name, "reel_count": None, "ok": False, "error": err,
                        "quality": None, "wall_seconds": round(total_wall, 2)}
            # re-run-once guard for the transient DB-drop flake
            if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
                count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
                total_wall += wall2
                if err2 is None and count2 is not None:
                    count, intervals = count2, intervals2  # the reproduced (or corrected)
value

            quality = None
            if gts and intervals is not None:
                row = rally_seg_eval.score_intervals(intervals, gts)
                quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")
                           if k in row}

        return {"name": name, "reel_count": count, "ok": True, "error": None,
                "quality": quality, "wall_seconds": round(total_wall, 2)}


    def load_manifest(path: str) -> Dict[str, Any]:
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
        """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
        from backend.config import load_config

        config = load_config()
        # apply manifest config overrides onto the typed config (dotted keys)
        for dotted, val in (manifest.get("config_overrides") or {}).items():
            _set_dotted(config, dotted, val)
        segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
        b_pv = (baseline or {}).get("per_video", {})
        results: List[Dict[str, Any]] = []
        total_wall = 0.0
        for v in manifest.get("videos", []):
            base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
            r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
```

```
            total_wall += float(r.get("wall_seconds") or 0.0)
            results.append(r)
        return results, total_wall


    def _set_dotted(config: Any, dotted: str, value: Any) -> None:
        """Best-effort dotted-path set onto a pydantic config (e.g. 'indexing.motion_threshold')."""
        parts = dotted.split(".")
        obj = config
        for p in parts[:-1]:
            obj = getattr(obj, p, None)
            if obj is None:
                return
        try:
            setattr(obj, parts[-1], value)
        except Exception:  # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't crash the run
            pass


    def _git_head() -> str:
        import subprocess
        try:
            out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
                                 capture_output=True, text=True, timeout=10)
            return out.stdout.strip() or "?"
        except Exception:  # noqa: BLE001
            return "?"


    def load_baseline(path: str) -> Optional[Dict[str, Any]]:
        if not os.path.exists(path):
            return None
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
        out = dict(snapshot)
        out["generated_at"] = _dt.date.today().isoformat()
        out["git_commit"] = _git_head()
        out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python scripts/nightly_reelcount.py "
                       "--update-baseline` after an INTENDED engine change or when adding a video. Tied to the "
                       "configured+checkpointed engine + the manifest corpus.")
        out.pop("cost", None)  # cost is per-run, not part of the frozen baseline
        os.makedirs(os.path.dirname(path), exist_ok=True)
        tmp = path + ".tmp"
        with open(tmp, "w", encoding="utf-8") as fh:
            json.dump(out, fh, indent=2, sort_keys=True)
            fh.write("\n")
        os.replace(tmp, path)


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count + quality + cost.")
        p.add_argument("--manifest", default=DEFAULT_MANIFEST)
        p.add_argument("--baseline", default=DEFAULT_BASELINE)
        p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
        p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
        p.add_argument("--vm-uptime-seconds", type=float, default=None,
                       help="authoritative VM uptime (boot?delete) for the cost line; "
```

```
                            "defaults to the in-process pipeline wall time (a lower bound)")
        p.add_argument("--no-rerun-guard", action="store_true",
                       help="disable the re-run-once-on-mismatch guard (the DB-drop flake
mitigation)")
        p.add_argument("--update-baseline", action="store_true",
                       help="freeze the current reel counts + quality as the new baseline
instead of checking")
        p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
        p.add_argument("--email-to", default=None, help="override the report recipient")
        p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
        p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
        return p


    def main(argv: Optional[List[str]] = None) -> int:
        import logging
        logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
        log = logging.getLogger("nightly_reelcount")
        args = build_parser().parse_args(argv)
        tols = Tolerances(quality_tol=args.quality_tol)
        date = args.date or _dt.date.today().isoformat()

        if not os.path.exists(args.manifest):
            log.error("manifest not found: %s", args.manifest)
            return 2
        manifest = load_manifest(args.manifest)
        baseline = load_baseline(args.baseline)

        try:
            run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
        except Exception as e:  # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
            log.error("run failed: %s", e)
            return 2
        if not run_results:
            log.error("no videos in manifest %s", args.manifest)
            return 2

        billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
        cost = cost_report(
            billed_seconds=billed,
            machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
            fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
            gpu_seconds=total_wall,
        )
        segmenter = manifest.get("segmenter", "motion")
        snapshot = summarize_results(run_results, segmenter, cost)
        snapshot["git_commit"] = _git_head()
        snapshot["generated_at"] = date

        if args.update_baseline:
            save_baseline(args.baseline, snapshot)
            log.info("baseline frozen ? %s", args.baseline)
            for r in run_results:
                log.info("  %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
            return 0

        if baseline is None:
```

```
514             log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
515             return 3
516
517         result = compare(baseline, snapshot, tols)
518         report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])
519
520         if not args.no_report:
521             os.makedirs(args.report_dir, exist_ok=True)
522             md_path = os.path.join(args.report_dir, f"{date}.md")
523             with open(md_path, "w", encoding="utf-8") as fh:
524                 fh.write(report + "\n")
525             with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526                 json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                            "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                           fh, indent=2, sort_keys=True)
529             log.info("report ? %s", md_path)
530
531         print(report)
532
533         if args.email:
534             try:
535                 from scripts.nightly_email import send_report
536                 subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537                 sent = send_report(subject, report, to=args.email_to)
538                 log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539             except Exception as e:  # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540                 log.error("email failed (report still written): %s", e)
541
542         code = result.exit_code()
543         if code == 1:
544             log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545         elif code == 4:
546             log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547         else:
548             log.info("nightly reel-count check PASSED  (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549         return code
550
551
552     if __name__ == "__main__":
553         raise SystemExit(main())
554
```

4. user (2026-06-22T09:45:40.267Z)

```
1       """Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret
Manager.
2
3       Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly).
Deliberately
4       dependency-light (stdlib ``smtplib`` + ``email.message``) and **graceful**: if no SMTP
password can
5       be resolved it logs + returns ``False`` (the report is still written/printed ? a broken
mailer must
6       never mask a regression).
7
```

```
8       Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:
9         ? Recipient:  ``--email-to`` arg  ?  ``NIGHTLY_EMAIL_TO``  ?  ``avi.dullu@gmail.com``
10        ? SMTP host:  ``NIGHTLY_SMTP_HOST``  (default ``smtp.gmail.com``)   · port
``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
11        ? SMTP user:  ``NIGHTLY_SMTP_USER``  (default = recipient)
12        ? SMTP pass:  ``NIGHTLY_SMTP_PASS``  ?  Secret Manager ``NIGHTLY_SMTP_SECRET``
13                      (``projects/<p>/secrets/<name>/versions/latest``; via the google-cloud-
secret-manager
14                       lib if installed, else the ``gcloud`` CLI). A Gmail **app password** is
the simplest.
15
16      Set the secret once (owner):  echo -n '<app-password>' | gcloud secrets create nightly-
smtp-pass --data-file=-
17      and on the VM:  export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-
pass/versions/latest
18                      export NIGHTLY_SMTP_USER=<your-gmail>
NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
19      """
20      from __future__ import annotations
21
22      import logging
23      import os
24      from email.message import EmailMessage
25      from typing import Optional
26
27      log = logging.getLogger("nightly_email")
28
29      DEFAULT_TO = "avi.dullu@gmail.com"
30      DEFAULT_HOST = "smtp.gmail.com"
31      DEFAULT_PORT = 587
32
33
34      def build_message(subject: str, body_md: str, from_addr: str, to_addr: str) ->
EmailMessage:
35          """A text/plain email carrying the markdown report (renders fine in any client).
Pure ? testable."""
36          msg = EmailMessage()
37          msg["Subject"] = subject
38          msg["From"] = from_addr
39          msg["To"] = to_addr
40          msg.set_content(body_md)
41          return msg
42
43
44      def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
45          """Resolve the SMTP password from env or GCP Secret Manager. Returns None if
unresolvable."""
46          env = env if env is not None else os.environ
47          direct = env.get("NIGHTLY_SMTP_PASS")
48          if direct:
49              return direct
50          secret_ref = env.get("NIGHTLY_SMTP_SECRET")
51          if not secret_ref:
52              return None
53          # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
54          try:
55              from google.cloud import secretmanager  # type: ignore
56              client = secretmanager.SecretManagerServiceClient()
57              return
client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
58          except Exception:  # noqa: BLE001 ? lib absent or auth issue ? try the CLI
59              pass
60          try:
61              import subprocess
62              name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in
```

```
secret_ref else secret_ref
63              out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--
secret", name],
64                                   capture_output=True, text=True, timeout=30)
65              return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else
None
66          except Exception as e:  # noqa: BLE001
67              log.warning("secret-manager resolution failed: %s", e)
68              return None
69
70
71      def send_report(subject: str, body_md: str, to: Optional[str] = None,
72                      env: Optional[dict] = None) -> bool:
73          """Send the report. Returns True if sent, False if skipped (no creds) ? never raises
on missing creds."""
74          env = env if env is not None else os.environ
75          to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
76          host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
77          port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
78          user = env.get("NIGHTLY_SMTP_USER") or to_addr
79          password = resolve_secret(env)
80          if not password:
81              log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ?
email skipped.")
82              return False
83
84          import smtplib
85          msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
86          with smtplib.SMTP(host, port, timeout=60) as s:
87              s.starttls()
88              s.login(user, password)
89              s.send_message(msg)
90          log.info("emailed report to %s", to_addr)
91          return True
92
```

5. assistant (2026-06-22T09:45:42.546Z)

Now let me read the other relevant files to get the full context:

6. user (2026-06-22T09:45:44.173Z)

```
1       #!/usr/bin/env bash
2       # Startup-script for the NIGHTLY engine-regression VM. Runs UNATTENDED on boot, then
DELETES the VM
3       # (cost rule: a running GPU VM is the only real cost). Survives SSH drops (it's a
startup-script, not
4       # an interactive session). Reads its config from instance metadata (set by launch.sh).
5       #
6       # Flow:  Ops Agent  ?  pull repo @ HEAD from GCS  ?  build venv + WASB env  ?  stage
weights  ?
7       #        run scripts/nightly_reelcount.py on the gs:// golden clips (emails the report)
?
8       #        upload report to GCS  ?  SELF-DELETE (even on failure, via the EXIT trap).
9       #
10      # Metadata keys (launch.sh sets these): repo-tgz-uri, git-sha, gcs-bucket, report-
prefix,
11      #   weights-uri, email (true/false). NOT verified in CI ? dry-run via launch.sh on a
real VM first.
12      set -uo pipefail
13      exec > >(tee -a /var/log/nightly-regression.log) 2>&1
14      echo "===== nightly-regression startup $(date -u +%FT%TZ) ====="
15      BOOT_EPOCH=$(date +%s)
16
17      md() { curl -s -H "Metadata-Flavor: Google"
```

```
"http://metadata.google.internal/computeMetadata/v1/$1"; }
18      NAME=$(md instance/name)
19      ZONE=$(md instance/zone | awk -F/ '{print $NF}')
20      REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
21      GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
22      BUCKET=$(md instance/attributes/gcs-bucket || true)
23      REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
24      WEIGHTS_URI=$(md instance/attributes/weights-uri || true)
25      DO_EMAIL=$(md instance/attributes/email || echo true)
26      SMTP_SECRET=$(md instance/attributes/smtp-secret || true)
27      SMTP_USER=$(md instance/attributes/smtp-user || true)
28      EMAIL_TO=$(md instance/attributes/email-to || echo avi.dullu@gmail.com)
29      DATE=$(date -u +%F)
30
31      # ALWAYS delete the VM on exit (success OR failure) ? never leave a billing GPU VM
behind.
32      cleanup() {
33        echo "===== self-delete $NAME ($ZONE) ====="
34        gcloud compute instances delete "$NAME" --zone="$ZONE" --quiet || \
35          echo "WARN: self-delete failed ? DELETE MANUALLY: gcloud compute instances delete
$NAME --zone=$ZONE -q"
36      }
37      trap cleanup EXIT
38
39      fail_email() {  # best-effort failure alert (the report email won't have fired on an
early crash)
40        echo "RUN FAILED ? attempting a failure alert"
41        if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
42          cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
43          NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
44          python - <<PY || true
45      from scripts.nightly_email import send_report
46      send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
47                  "The nightly engine-regression run failed on the VM before producing a
report.\n"
48                  "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial
log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
49      PY
50        fi
51      }
52      trap 'fail_email' ERR
53
54      # 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
55      curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
56        sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent
install failed (non-fatal)"
57
58      # 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the
worker SA).
59      mkdir -p ~/rally && cd ~/rally
60      gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
61      tar -xzf /tmp/rally.tgz -C ~/rally
62      echo "$GIT_SHA" > ~/rally/GIT_SHA
63
64      # 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per
deploy/gcp/README §4).
65      sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
66      export TMPDIR=/scratch 2>/dev/null || true
67      ./deploy/digitalocean/bootstrap.sh --gpu
68      . .venv/bin/activate
69      pip install -e '.[storage-gcs]'
70      bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned
nonzero ? continuing"
71
```

```
72      # 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4).
73      if [ -n "$WEIGHTS_URI" ]; then
74        mkdir -p ~/models/WASB-SBDT/pretrained_weights
75        gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
76        export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
77      fi
78
79      # 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
uptime so far.
80      UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
81      EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
82      set +e
83      NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
84        python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
85      RUN_RC=$?
86      set -e
87      echo "runner exit code: $RUN_RC  (0 pass / 1 regression / 4 stale)"
88
89      # 6) Upload the dated report to GCS for the record (alongside the email).
90      if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
91        gcloud storage cp -r "output/nightly_reelcount/$DATE."*
"gs://$BUCKET/$REPORT_PREFIX/$DATE/" || \
92          gcloud storage cp -r output/nightly_reelcount "gs://$BUCKET/$REPORT_PREFIX/$DATE/"
|| true
93      fi
94
95      echo "===== nightly-regression done rc=$RUN_RC $(date -u +%FT%TZ) ====="
96      trap - ERR   # disable the failure-email trap; the EXIT (self-delete) trap stays
97      # the EXIT trap deletes the VM
98
```

7. user (2026-06-22T09:45:44.935Z)

```
1       {
2         "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling.",
3         "segmenter": "wasb_hybrid",
4         "detector_impl": "native",
5         "config_overrides": {
6           "indexing.default_segmenter": "wasb_hybrid",
7           "indexing.detector_impl": "native"
8         },
9         "machine_usd_per_hr": 0.70,
10        "fx_inr_per_usd": 83.0,
11        "videos": [
12          {
13            "name": "GX030094",
14            "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX030094.mp4",
15            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX030094_golden_features.json"
16          },
17          {
18            "name": "Badminton_BXH_2",
19            "uri": "gs://khelsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
20            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
21          },
```

```
22          {
23            "name": "GX010128",
24            "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX010128.mp4",
25            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX010128_golden_features.json"
26          }
27        ]
28      }
29
```

8. assistant (2026-06-22T09:45:47.787Z)

Now let me examine the exception handler more carefully around lines 485-489 and the surrounding context:

9. user (2026-06-22T09:45:49.120Z)

```
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e:  # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                     "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                     "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341             random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342             db = Database(db_path)
343             video_id = compute_video_id(local)
344             t0 = time.monotonic()
345             result = seg_cls(db, config).process_video(video_id, local)
346             wall = time.monotonic() - t0
347             if isinstance(result, Failure):
348                 return None, None, wall, getattr(result, "message", "segmenter failed")
349             segs = result.value if hasattr(result, "value") else result
350             intervals = _segments_to_intervals(segs)
351             return len(segs), intervals, wall, None
352
353         import tempfile
354         total_wall = 0.0
355         with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356             count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357             total_wall += wall
358             if err is not None:
359                 return {"name": name, "reel_count": None, "ok": False, "error": err,
360                         "quality": None, "wall_seconds": round(total_wall, 2)}
361             # re-run-once guard for the transient DB-drop flake
362             if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363                 count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
```

```
                total_wall += wall2
                if err2 is None and count2 is not None:
                    count, intervals = count2, intervals2  # the reproduced (or corrected)
value

            quality = None
            if gts and intervals is not None:
                row = rally_seg_eval.score_intervals(intervals, gts)
                quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")
                           if k in row}

        return {"name": name, "reel_count": count, "ok": True, "error": None,
                "quality": quality, "wall_seconds": round(total_wall, 2)}


    def load_manifest(path: str) -> Dict[str, Any]:
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
        """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
        from backend.config import load_config

        config = load_config()
        # apply manifest config overrides onto the typed config (dotted keys)
        for dotted, val in (manifest.get("config_overrides") or {}).items():
            _set_dotted(config, dotted, val)
        segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
        b_pv = (baseline or {}).get("per_video", {})
        results: List[Dict[str, Any]] = []
        total_wall = 0.0
        for v in manifest.get("videos", []):
            base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
            r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
            total_wall += float(r.get("wall_seconds") or 0.0)
            results.append(r)
        return results, total_wall


    def _set_dotted(config: Any, dotted: str, value: Any) -> None:
        """Best-effort dotted-path set onto a pydantic config (e.g.
'indexing.motion_threshold')."""
        parts = dotted.split(".")
        obj = config
        for p in parts[:-1]:
            obj = getattr(obj, p, None)
            if obj is None:
                return
        try:
            setattr(obj, parts[-1], value)
        except Exception:  # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
crash the run
            pass


    def _git_head() -> str:
        import subprocess
        try:
            out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
                                 capture_output=True, text=True, timeout=10)
```

```
423             return out.stdout.strip() or "?"
424         except Exception:  # noqa: BLE001
425             return "?"
426 
427 
428     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
429         if not os.path.exists(path):
430             return None
431         with open(path, encoding="utf-8") as fh:
432             return json.load(fh)
433 
434 
435     def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
436         out = dict(snapshot)
437         out["generated_at"] = _dt.date.today().isoformat()
438         out["git_commit"] = _git_head()
439         out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python
scripts/nightly_reelcount.py "
440                        "--update-baseline` after an INTENDED engine change or when adding a
video. Tied to the "
441                        "configured+checkpointed engine + the manifest corpus.")
442         out.pop("cost", None)  # cost is per-run, not part of the frozen baseline
443         os.makedirs(os.path.dirname(path), exist_ok=True)
444         tmp = path + ".tmp"
445         with open(tmp, "w", encoding="utf-8") as fh:
446             json.dump(out, fh, indent=2, sort_keys=True)
447             fh.write("\n")
448         os.replace(tmp, path)
449 
```

10. assistant (2026-06-22T09:45:51.119Z)

Now I need to check the actual exception handling in the main function and understand the issue properly:

11. user (2026-06-22T09:45:52.476Z)

```
330:    except Exception as e:  # noqa: BLE001 ? surface, never hide
414:    except Exception:  # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't crash
the run
424:    except Exception:  # noqa: BLE001
487:    except Exception as e:  # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
539:        except Exception as e:  # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
```

12. assistant (2026-06-22T09:45:56.105Z)

Let me trace through the code logic more carefully. The key question is: what exceptions can be raised INSIDE `run_corpus()` and how are they handled?

13. user (2026-06-22T09:45:56.787Z)

```
def main(argv: Optional[List[str]] = None) -> int:
    import logging
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
    log = logging.getLogger("nightly_reelcount")
    args = build_parser().parse_args(argv)
    tols = Tolerances(quality_tol=args.quality_tol)
    date = args.date or _dt.date.today().isoformat()

    if not os.path.exists(args.manifest):
        log.error("manifest not found: %s", args.manifest)
        return 2
```

```
    manifest = load_manifest(args.manifest)
    baseline = load_baseline(args.baseline)

    try:
        run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
    except Exception as e:  # noqa: BLE001 ? operational failure (deps/config) ? exit 2, never a
false "pass"
        log.error("run failed: %s", e)
        return 2
    if not run_results:
        log.error("no videos in manifest %s", args.manifest)
        return 2

    billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else total_wall
    cost = cost_report(
        billed_seconds=billed,
        machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
        fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
        gpu_seconds=total_wall,
    )
    segmenter = manifest.get("segmenter", "motion")
    snapshot = summarize_results(run_results, segmenter, cost)
    snapshot["git_commit"] = _git_head()
    snapshot["generated_at"] = date

    if args.update_baseline:
        save_baseline(args.baseline, snapshot)
        log.info("baseline frozen ? %s", args.baseline)
        for r in run_results:
            log.info("  %s: reels=%s ok=%s", r["name"], r.get("reel_count"), r.get("ok"))
        return 0

    if baseline is None:
        log.error("no baseline at %s ? freeze one with --update-baseline (on the configured
engine).", args.baseline)
        return 3

    result = compare(baseline, snapshot, tols)
    report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])

    if not args.no_report:
        os.makedirs(args.report_dir, exist_ok=True)
        md_path = os.path.join(args.report_dir, f"{date}.md")
        with open(md_path, "w", encoding="utf-8") as fh:
            fh.write(report + "\n")
        with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8") as fh:
            json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
                       "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
                      fh, indent=2, sort_keys=True)
        log.info("report ? %s", md_path)

    print(report)

    if args.email:
        try:
            from scripts.nightly_email import send_report
            subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count engine
regression {date}"
            sent = send_report(subject, report, to=args.email_to)
            log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
```

```
        except Exception as e:  # noqa: BLE001 ? a broken mailer must NOT mask the regression
verdict
            log.error("email failed (report still written): %s", e)

    code = result.exit_code()
    if code == 1:
        log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
    elif code == 4:
        log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
    else:
        log.info("nightly reel-count check PASSED  (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
    return code
```

14. assistant (2026-06-22T09:45:59.098Z)

Now let me trace what happens in `run_one_video` if an exception occurs that's NOT caught inside it:

15. user (2026-06-22T09:46:00.472Z)

```
def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
                  rerun_on_mismatch: bool = True,
                  baseline_count: Optional[int] = None) -> Dict[str, Any]:
    """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy backend
imports.

    The re-run-once guard (the one known non-determinism ? a transient ``add_play_segment``?None
drop,
    database.py): if the count != the frozen baseline, re-process the video ONCE; a transient
drop won't
    reproduce, a real change will."""
    import random
    import time

    from backend.eval import rally_seg_eval
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters import segmenter_registry
    from backend.results import Failure
    from backend.storage import fetch
    from backend.utils.hashing import compute_video_id

    name = video["name"]
    try:
        local = fetch(video["uri"])
    except Exception as e:  # noqa: BLE001 ? surface, never hide
        return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed: {e}",
                "quality": None, "wall_seconds": 0.0}

    gts = _load_gts(video.get("labels_uri"))
    seg_cls = segmenter_registry.get(segmenter)
    if not seg_cls:
        return {"name": name, "reel_count": None, "ok": False,
                "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}

    def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float, float]]],
float, Optional[str]]:
        random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields are
reproducible too
        db = Database(db_path)
        video_id = compute_video_id(local)
        t0 = time.monotonic()
```

```
        result = seg_cls(db, config).process_video(video_id, local)
        wall = time.monotonic() - t0
        if isinstance(result, Failure):
            return None, None, wall, getattr(result, "message", "segmenter failed")
        segs = result.value if hasattr(result, "value") else result
        intervals = _segments_to_intervals(segs)
        return len(segs), intervals, wall, None

    import tempfile
    total_wall = 0.0
    with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
        count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
        total_wall += wall
        if err is not None:
            return {"name": name, "reel_count": None, "ok": False, "error": err,
                    "quality": None, "wall_seconds": round(total_wall, 2)}
        # re-run-once guard for the transient DB-drop flake
        if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
            count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
            total_wall += wall2
            if err2 is None and count2 is not None:
                count, intervals = count2, intervals2  # the reproduced (or corrected) value

        quality = None
        if gts and intervals is not None:
            row = rally_seg_eval.score_intervals(intervals, gts)
            quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25", "tol_precision",
"tol_recall")
                       if k in row}

    return {"name": name, "reel_count": count, "ok": True, "error": None,
            "quality": quality, "wall_seconds": round(total_wall, 2)}
```

16. assistant (2026-06-22T09:46:02.984Z)

Now I need to analyze the code flow very carefully. Let me identify the potential exception scenarios:

17. user (2026-06-22T09:46:15.241Z)

```
=== EXCEPTION FLOW ANALYSIS ===

1. run_corpus() calls run_one_video() in a loop (line 398)
   for v in manifest.get('videos', []):
       r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)

2. run_one_video() has these exception handlers:
   - Line 328-332: fetch(video['uri']) ? caught, returns {'ok': False, 'error': '...'}
   - Line 334: _load_gts() called but NOT wrapped in try/except
   - Line 335: segmenter_registry.get() called but NOT wrapped in try/except
   - Line 340-351: _process() nested function called in tempfile context
     - Inside _process(): Database(db_path) ? NOT wrapped
     - seg_cls(db, config).process_video() ? NOT wrapped (returns Failure object)
     - rally_seg_eval.score_intervals() at line 370 ? NOT wrapped

3. In main(), line 485-489:
   try:
       run_results, total_wall = run_corpus(...)
   except Exception as e:
       return 2  # operational failure

4. Key insight:
   - If run_one_video() calls _load_gts() and it raises an exception
     (e.g., missing JSON file, permission denied, malformed CSV)
     that exception is NOT caught in run_one_video()
```

```
   - It propagates up to run_corpus() which also doesn't catch it
   - It then hits the try/except at line 485-489 in main()
   - This returns exit code 2 ('operational failure')

5. The concern (from the finding):
   - Exit code 2 is for 'operational failure' (missing config, import error, GPU OOM)
   - This is INTENTIONAL per the docstring and design
   - The concern: if a REGRESSION happens (e.g., segmenter determinism bug)
     but also triggers an exception (e.g., quality metric calculation crashes)
     the exception masking would hide the regression

6. Reality check:
   - Most exception points are already protected by error handlers
   - Unprotected: _load_gts (but it's optional), score_intervals (metric calc)
   - A determinism bug would more likely cause wrong reel counts (caught)
     not an exception during metric scoring
```

18. assistant (2026-06-22T09:46:18.434Z)

```
Let me examine the exit codes more carefully in the docstring:
```

19. user (2026-06-22T09:46:19.903Z)

```
"""Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM nightly).

Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a deployment
serves
it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does not
change**
vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score of
CACHED
trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real model*,
so it
needs a GPU ? hence it runs on an ephemeral GCP GPU VM (``deploy/gcp/nightly_regression/``),
emails the
owner the report, and self-deletes. It is **default-additive**: a tripwire, not the promotion
gate.

What it asserts / reports, per the owner's ask ("quality metrics + regression report + cost"):
  ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected highlight
clips.
    Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so this is
asserted
    **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-None, see
§guard).
  ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match block
from
    ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
  ? COST of the run ? VM uptime (or processing wall) × machine $/hr via ``backend.billing`` ?
USD + INR.

Extensible pattern (add a video = one manifest row + re-freeze):
  ``eval_baselines/reelcount_manifest.json``  ? the videos (gs:// / local / gdrive:// URIs) +
segmenter
  ``eval_baselines/reelcount_baseline.json``  ? the frozen expected counts (regenerated, never
hand-edited)

Usage
-----
Freeze the baseline from the current configured+checkpointed engine (after an intended change,
or first run):
    python scripts/nightly_reelcount.py --update-baseline

Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails if
```

```
--email):
    python scripts/nightly_reelcount.py --email

Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality dropped) · 2
operational
(manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter changed ? re-
freeze).
"""
```

20. assistant (2026-06-22T09:46:22.459Z)

```
Now I need to analyze the actual risk. The key question is: **does the exception handler at line
485-489 cause a REAL defect in production?**

Let me check what happens:
```

21. user (2026-06-22T09:46:39.407Z)

```
=== ADVERSARIAL VERIFICATION ===

CLAIM: Exception handler at line 485-489 will mask real exceptions if
       they occur BEFORE run_corpus returns.

REALITY CHECK:

1. When does an exception propagate out of run_corpus()?
   - run_corpus() iterates over videos and calls run_one_video()
   - run_one_video() is designed to catch most errors and return {'ok': False, 'error': '...'}
   - If an UNcaught exception occurs inside run_one_video(), it propagates up

2. What UNcaught exceptions are possible in run_one_video()?
   - Line 334: _load_gts(video.get('labels_uri'))
   - Line 335: segmenter_registry.get(segmenter)
   - Line 342: Database(db_path)
   - Line 345: seg_cls(db, config).process_video(...)
   - Line 370: rally_seg_eval.score_intervals(...)

3. Is this INTENTIONAL?
   - Line 487 comment says: 'operational failure (deps/config) ? exit 2, never a false pass'
   - Docstring line 30-31 confirms exit 2 is for 'operational' failures
   - Design intent: missing deps, import errors, GPU OOM ? exit 2

4. The REAL question: Is this the RIGHT design?
   - If a segmenter CRASHES with a determinism bug during run_corpus()
     ? Exception ? Caught at line 487 ? exit 2 ('operational failure')
   - But it SHOULD be exit 1 ('regression') because the video failed

5. The counter-argument (from the finding detail):
   - A determinism bug causes WRONG REEL COUNTS ? caught by the count check
   - A CRASH during segmentation would be a deep bug (segmenter, not regression)
   - Line 330 catches fetch errors
   - Line 338 checks segmenter exists
   - Line 347-348 checks for Failure returns

VERDICT on the finding:

TRUE: An exception in run_corpus() returns exit 2, not 1
TRUE: This would mask a regression detection if combined with a crash

BUT: The design is INTENTIONAL (line 487 comment)
AND: run_one_video() is designed to catch predictable errors
AND: Unpredictable crashes are operational failures (deps, import, OOM)
AND: This is a NIGHTLY TRIPWIRE (not promotion gate) so operational failures
     are acceptable as 'exit 2' (needs investigation, not a regression)
```

```
SEVERITY: This is NOT a defect in typical operation.

SCENARIO where it COULD matter:
  - Segmenter.__init__() raises an exception (import bug in motion.py)
  - Segmenter is the CHANGED code (real regression)
  - Exception is caught at line 487
  - Exit 2 is returned (operational failure)
  - The regression is NOT detected

But that's misclassified as operational when it's a deployment issue.
The risk is REAL but RARE (requires both crash + changed code).
```

22. assistant (2026-06-22T09:46:41.549Z)

```
Let me check if there's any video error handling in the loop structure:
```

23. user (2026-06-22T09:46:42.842Z)

```
def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
               rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
    """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
    from backend.config import load_config

    config = load_config()
    # apply manifest config overrides onto the typed config (dotted keys)
    for dotted, val in (manifest.get("config_overrides") or {}).items():
        _set_dotted(config, dotted, val)
    segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
    b_pv = (baseline or {}).get("per_video", {})
    results: List[Dict[str, Any]] = []
    total_wall = 0.0
    for v in manifest.get("videos", []):
        base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
        r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
        total_wall += float(r.get("wall_seconds") or 0.0)
        results.append(r)
    return results, total_wall
```

24. assistant (2026-06-22T09:46:45.521Z)

```
Now I understand the issue completely. Let me verify the claim is **FALSE** by checking one more
critical thing - whether `_load_gts` is likely to raise an exception:
```

25. user (2026-06-22T09:46:46.444Z)

```
272     def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
273         """Load ground-truth rally intervals from a golden-features JSON (``gts`` key) or a
rallies CSV
274         (``start,end`` columns / pairs). Returns None when no labels are configured."""
275         if not labels_ref:
276             return None
277         from backend.storage import fetch
278         local = fetch(labels_ref)
279         if local.endswith(".json"):
280             with open(local, encoding="utf-8") as fh:
281                 data = json.load(fh)
282             return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
283         # CSV: header start,end (or first two numeric columns)
284         import csv
285         out: List[Tuple[float, float]] = []
286         with open(local, encoding="utf-8") as fh:
287             for row in csv.reader(fh):
```

```
288                 try:
289                     out.append((float(row[0]), float(row[1])))
290                 except (ValueError, IndexError):
291                     continue
292         return out
293
294
295     def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
296         """``(start, end)`` pairs from segmenter result dicts, tolerant of the two key
conventions in the
```

26. assistant (2026-06-22T09:46:55.064Z)

Perfect! Now I can make a final call. The issue is:

1. `_load_gts()` at line 278 calls `fetch(labels_ref)` which can raise an exception
2. `_load_gts()` at line 281 calls `json.load()` which can raise JSONDecodeError
3. Neither of these is wrapped in try/except in `run_one_video()`
4. If they raise, the exception propagates to `run_corpus()`
5. The loop in `run_corpus()` (lines 396-400) has NO try/except around the `run_one_video()` call
6. The exception then propagates to `main()`
7. At line 485-489, it's caught and returns exit 2 (operational failure)

However, the **finding's claim is FALSE** because:
- The docstring and code design **intentionally** treats any exception from `run_corpus()` as operational failure (exit 2)
- This is documented explicitly in the comment at line 487
- The design distinguishes between "regression detection failed because pipeline crashed" (exit 2) vs "regression detected" (exit 1)
- This is APPROPRIATE for a TRIPWIRE nightly (not the promotion gate)
- A determinism bug would manifest as WRONG REEL COUNTS (line 197-198), not an exception
- The scenario described (segmenter crash + changed code + determinism bug) is extremely unlikely and would still be caught as operational failure (which is correct for a broken deployment)

Let me verify there's no wrap-around logic that might hide this:

27. user (2026-06-22T09:46:55.972Z)

```
689c738 docs(pkg): mark P2c ? + the loop closed ? v1 LIGHT is feature-complete (#313)
293e5c0 feat(pkg): re-bundle backend/ui ? serve the P2c first-run wizard from indexer --app
(#312)
f8b2c3b feat(pkg): P2b-install ? batteries-included install + truly-local 'motion' first-run
default (gate O8) (#310)
0ed4cc6 fix(segmenter): fail the job on an unrunnable config instead of a silent 0-rally success
(Finding B) (#311)
a42c132 docs(pkg): track the batteries-included app plan (PACKAGING_PLAN.md) + index pointers
(#309)
de2bf14 feat(pkg): P2e ? lazy-torch so the LIGHT tier imports with NO torch installed (default-
OFF) (#308)
679e642 feat(pkg): P2d ? DB upgrade-safety (downgrade guard + backup-before-migrate + export)
(default-OFF) (#307)
f01dc85 feat(db): v7 ? record the UI locale per job (PMF analytics) + count_jobs_by_locale
(#306)
fa7ee3f feat(pkg): P2a ? single ffmpeg/ffprobe resolver (env-overridable; one fail-loud place)
(default-OFF) (#305)
e99d0c3 feat(stitch): localize the reel watermark to the UI locale (+ OFL Noto fonts) (#304)
ca34787 feat(pkg): P2b ? friendly `indexer --app` launcher (boot localhost server + open the
bundled UI) (#303)
4a06b8b feat(pkg): P1b ? bundle the KhelSutra SPA single-origin into the engine wheel (default-
OFF) (#302)
be0c434 feat(pkg): P1a ? net-new console/wizard endpoints: /api/version, /api/capabilities,
/api/videos, /api/reels (#300)
cb7394b feat(serving): T11 ? retention sweep for stale uploads + caches (#299)
```

```
c039b90 feat(pkg): P0e ? single-instance lock so a 2nd process can't corrupt in-flight job state
(default-OFF) (#298)
e8bad40 feat(pkg): P0d ? declare scipy (undeclared serving dep) + license-scan & built-wheel CI
lanes (#297)
3895963 feat(pkg): P0c ? config/profile-driven bind address + DNS-rebinding Host-header guard
(default-OFF) (#296)
fc78fce feat(pkg): P0b ? redact secret-shaped fields from the unauthenticated GET /api/config
(#295)
0c0e064 feat(serving): exposure-safety boot posture ? fail fast on a half-configured exposed
engine (default-OFF) (#294)
82a28f4 feat(pkg): P0a ? resolved app-data HOME (RALLY_APP_HOME) for config/.env/DB/output
(default-OFF) (#293)
```

28. assistant (2026-06-22T09:46:58.022Z)

Let me search for any recent changes to nightly_reelcount.py:

29. user (2026-06-22T09:46:58.993Z)

(Bash completed with no output)

30. assistant (2026-06-22T09:47:01.060Z)

Now let me make the final determination by examining whether the behavior is truly a defect:

31. user (2026-06-22T09:47:23.741Z)

```
=== FINAL VERDICT ===

FINDING CLAIM:
  Exception handler at line 485 will mask real exceptions
  if they occur before run_corpus returns.

EVIDENCE FROM CODE:

1. Line 485-489 (main):
   try:
       run_results, total_wall = run_corpus(...)
   except Exception as e:
       log.error('run failed: %s', e)
       return 2  # exit 2 = operational failure

2. Exit code meanings (docstring line 30-31):
   0 pass
   1 regression (reel-count changed / a video errored / quality dropped)
   2 operational (manifest/videos missing, nothing ran)
   3 no baseline
   4 stale (corpus/segmenter changed)

3. Possible exceptions in run_one_video (UNCAUGHT):
   - Line 278: _load_gts() -> fetch() can raise on missing/permission
   - Line 281: json.load() can raise on malformed JSON
   - Line 342: Database(db_path) constructor
   - Line 345: seg_cls(...).process_video() - returns Failure, doesn't raise
   - Line 370: rally_seg_eval.score_intervals() - unlikely to raise

4. When run_one_video() raises an exception:
   -> propagates to run_corpus() (no catch)
   -> propagates to main() try block at line 485
   -> caught at line 487
   -> returns 2 (operational failure)

CRITICAL ANALYSIS:
```

```
A) Is the finding technically TRUE?
   YES - An exception DOES cause exit 2, not proper error handling

B) Is it a REAL PRODUCTION DEFECT?
   NO - Because:

   1. DESIGN IS INTENTIONAL (line 487 comment):
      'operational failure (deps/config) ? exit 2, never a false pass'

   2. The regression detection logic is:
      - Reel counts must be exact (line 197-198)
      - A video can fail with ok=False (line 191-194)
      - Quality metrics checked with tolerance (line 201-208)

   3. Run-time exceptions (JSON parsing, DB init) ARE operational:
      - They indicate deployment/config issues, not regressions
      - Example: corrupt golden labels file = operational, not a regression

   4. A determinism regression would manifest as:
      - DIFFERENT reel counts (caught by exact comparison at line 197)
      - NOT an exception (unless segmenter code is completely broken)

   5. This is a TRIPWIRE NIGHTLY (not promotion gate):
      - Exit 2 means 'investigate this run', which is correct
      - Prevents false passes (line 487 'never a false pass')

C) COULD this mask a REAL regression?
   Scenario: Segmenter code is rewritten and crashes on init
   - Exception caught -> exit 2 (operational failure)
   - Owner sees exit 2 in the email, investigates
   - Finds import error or regression in segmenter init

   This is NOT a defect - it's correct behavior.
   A crashing segmenter IS an operational failure.

D) Is the comment at line 487 accurate?
   YES - 'never a false pass' is the key guarantee:
   - Exit 0 = regression checks passed
   - Exit 1 = regression detected
   - Exit 2 = couldn't run (operational issue)
   - None of these is a false 'pass' (exit 0)

CONCLUSION:
------------------------------------------------------------
The finding is REAL=FALSE

Reason: While the exception handler DOES catch exceptions from
run_corpus(), this is INTENTIONAL DESIGN, not a defect:

- Operational failures (JSON parse, missing deps) -> exit 2
- This prevents false positives (line 487 'never a false pass')
- Regressions are caught by exact reel-count comparison
- A determinism bug would manifest as wrong counts, not exceptions
- The nightly owner receives exit 2 and knows to investigate

The design is SOUND for a tripwire nightly regression test.
```

32. assistant (2026-06-22T09:47:27.259Z)

Perfect. Now I have all the evidence I need. Let me return the structured output:

33. user (2026-06-22T09:47:35.176Z)

Structured output provided successfully

34. assistant (2026-06-22T09:47:42.398Z)

Summary

I've analyzed the claimed finding about the exception handler in `scripts/nightly_reelcount.py` (lines 485-489) and conducted an adversarial verification of whether it truly masks regression detection.

Verdict: REAL = FALSE

The finding correctly observes that exceptions from `run_corpus()` are caught and converted to exit code 2 (operational failure). However, this is **intentional design**, not a defect:

1. **Design is explicitly documented**: Line 487 comment states "operational failure (deps/config) ? exit 2, never a false pass"

2. **Regressions are detected by exact reel-count comparison** (line 197-198), not by exception handling. A determinism bug would produce different counts, not an exception.

3. **Operational failures (JSON parsing, missing deps, DB init errors) are correctly routed to exit 2**, which signals the nightly owner to investigate deployment/configuration issues.

4. **The guarantee "never a false pass" is upheld**: Exit 0 (pass) is only returned when regression checks actually pass. Exceptions don't cause false negatives.

5. **This is a tripwire nightly** (not the promotion gate), so exit 2 behavior is appropriate ? it means "investigate this run" rather than "regression detected."

For a scenario where a determinism bug also causes an exception during segmentation, that would correctly be caught as an operational failure requiring investigation, which is the right outcome.